

Universidad del Valle de Guatemala

Licenciatura en Ingeniería en Ciencias de la Computación

Construcción de Compiladores – CC-3032, sección 10

Laboratorio 01

Uso de generadores de analizadores léxicos y sintácticos

Analizador de Compiscript

Documentación teórica y práctica

Catedrático: Ingeniero Carlos Valdez

Grupo: DragonsSlayers 2.0

Integrantes: Pablo Daniel Barillas Moreno – 22193

Hugo Daniel Barillas Ajín – 23556

Ernesto Ascencio – 23009

Guatemala, 2026

Resumen

Este documento presenta la fundamentación, diseño, implementación y validación del Laboratorio 01 de Construcción de Compiladores. El producto desarrollado es una herramienta que analiza archivos de Compiscript con extensión `.cps`. La solución utiliza una gramática combinada de ANTLR 4 y el generador `antlr4ts-cli` para producir un lexer y un parser TypeScript reales. El mismo núcleo se consume desde una interfaz web construida con React y Vite, una aplicación de consola y una aplicación de escritorio empaquetada con Electron.

La herramienta fuerza la tokenización completa, conserva los tokens reconocibles después de errores léxicos, ejecuta el parser desde la regla `program` y aplica `DefaultErrorStrategy` para recuperarse de errores sintácticos. Los diagnósticos se normalizan al español e incluyen fase, línea, columna y símbolo relacionado. También se agrupan caracteres inválidos contiguos, se eliminan duplicados, se filtran cascadas derivadas y se limita la salida a 100 diagnósticos por fase sin detener el procesamiento.

El resultado incluye tabla de tokens, errores separados por fase, resumen de aceptación y árbol de parseo textual y visual. La suite automatizada contiene 24 pruebas que cubren construcciones válidas, recuperación ante múltiples errores, cadenas incorrectas, sincronización de ejemplos, límites de diagnósticos y los ocho casos de complejidad baja y media definidos por la rúbrica. El alcance se limita deliberadamente a análisis léxico y sintáctico: no hay análisis semántico, interpretación ni generación de código.

Palabras clave: compiladores, análisis léxico, análisis sintáctico, ANTLR, Compiscript, TypeScript, recuperación de errores.

Índice

Resumen	i
1 Introducción	1
1.1 Objetivo general	1
1.2 Objetivos específicos	2
1.3 Alcance y exclusiones	2
2 Fundamentos teóricos	3
2.1 De caracteres a estructura	3
2.2 Análisis léxico	3
2.3 Gramáticas libres de contexto	4
2.4 Precedencia y asociatividad	4
2.5 ANTLR 4 y Adaptive LL(*)	5
2.6 Recuperación de errores	5
2.6.1 Propagación y cascadas de errores	5
3 Especificación de Compiscript	6
3.1 Unidades léxicas	6
3.2 Declaraciones y tipos	6
3.3 Funciones y clases	7
3.4 Control de flujo	7
3.5 Accesos y expresiones	7
4 Arquitectura y diseño	8

4.1	Arquitectura general	8
4.2	Generación e integración	9
4.3	Flujo en tiempo de ejecución	10
4.4	Modelo de resultado	11
5	Implementación práctica	12
5.1	Organización del repositorio	12
5.2	Gramática combinada	13
5.3	Integración en <code>analyze.ts</code>	14
5.4	Recolección y normalización de errores	15
5.5	Representación del árbol	16
5.6	Interfaz web	17
5.7	Compatibilidad con navegador	17
5.8	Interfaz de consola	17
5.9	Aplicación de escritorio	18
6	Pruebas y validación	19
6.1	Estrategia de pruebas	19
6.2	Casos demostrativos	20
6.3	Matriz de casos exigidos por la rúbrica	20
6.4	Pruebas de distribución y recuperación de errores	21
6.4.1	Caso exclusivamente léxico	22
6.4.2	Caso exclusivamente sintáctico	22
6.4.3	Caso léxico y sintáctico alternado	22
6.5	Comandos de verificación	22
7	Instalación, ejecución y distribución	23
7.1	Requisitos de desarrollo	23
7.2	Ejecución web	23
7.3	Uso de la aplicación	24
7.4	Uso de la CLI	24

7.5 Construcción de ejecutables	24
8 Cumplimiento del laboratorio	25
9 Resultados, limitaciones y trabajo futuro	26
9.1 Resultados obtenidos	26
9.2 Limitaciones actuales	26
9.3 Trabajo futuro	27
10 Conclusiones	28
Referencias	28
A Preguntas interesantes a las cuales se responde	30
B Inventario de comandos	31
C Ficha técnica del producto	32

Índice de figuras

4.1	Arquitectura de reutilización del núcleo de análisis.	8
4.2	Flujo de generación de analizadores.	9
4.3	Pipeline de análisis y bifurcación por modo.	10
5.1	Proceso de normalización de diagnósticos.	16
5.2	Ejemplo simplificado de árbol de parseo.	16

Índice de cuadros

2.1	Precedencia de expresiones, de menor a mayor.	4
3.1	Familias principales de tokens.	6
5.1	Normalización de mensajes de ANTLR.	15
5.2	Códigos de salida de la CLI.	17
6.1	Cobertura automatizada del proyecto.	19
6.2	Resultados verificados de la matriz de la rúbrica.	21
6.3	Resultados verificados de los casos de distribución de errores.	21
8.1	Trazabilidad entre requisitos e implementación.	25

Capítulo 1

Introducción

Un compilador o intérprete necesita transformar texto plano en una representación estructurada antes de poder comprobar significados o ejecutar instrucciones. Las dos primeras etapas de esta transformación son el análisis léxico y el análisis sintáctico. El primero agrupa caracteres en unidades llamadas *tokens*; el segundo determina si la secuencia de tokens puede derivarse de una gramática formal.

El laboratorio estudia estas etapas mediante una herramienta generadora. En lugar de programar manualmente una máquina de estados y un parser, el lenguaje se describe en un archivo de gramática: `Compiscript.g4`. ANTLR genera las clases que reconocen la entrada y la aplicación se ocupa de coordinarlas, recolectar resultados, mejorar los mensajes y presentarlos de forma útil.

Compiscript es un lenguaje educativo inspirado en TypeScript. Incluye declaraciones, tipos básicos, arreglos, expresiones con precedencia, funciones, clases y estructuras de control. La aplicación permite cargar código propio o usar tres casos demostrativos integrados. Adicionalmente, `examples/rubric` aporta ocho entradas reproducibles que cubren los cuatro escenarios de complejidad baja y los cuatro de complejidad media solicitados en la evaluación.

1.1 Objetivo general

Diseñar e implementar un analizador léxico y sintáctico para archivos Compiscript mediante un generador de analizadores, con una interfaz que muestre tokens, diagnósticos comprensibles y el árbol de parseo sin detenerse en el primer error recuperable.

1.2 Objetivos específicos

- Definir la sintaxis concreta de Compiscript en una gramática ANTLR.
- Generar e integrar un lexer y un parser TypeScript a partir de esa gramática.
- Mostrar tokens con tipo, lexema, posición y canal.
- Distinguir errores léxicos de errores sintácticos y traducir los mensajes técnicos más comunes a español.
- Continuar el análisis ante entradas incorrectas y controlar las cascadas de diagnósticos.
- Visualizar y exportar los resultados desde una interfaz gráfica.
- Ofrecer ejecución alternativa mediante CLI y empaquetado de escritorio.
- Verificar el comportamiento mediante pruebas automatizadas y casos `.cps`.

1.3 Alcance y exclusiones

El proyecto reconoce la estructura léxica y sintáctica del lenguaje. Un archivo se considera aceptado cuando las listas de errores léxicos y sintácticos están vacías. El árbol producido demuestra qué estructura reconoció el parser.

No se construye una tabla de símbolos, no se resuelven nombres, no se comprueban tipos, no se evalúan expresiones y no se genera código intermedio, máquina virtual o ensamblador. Por ejemplo, una asignación entre tipos incompatibles puede ser sintácticamente válida aunque sería rechazada en una futura fase semántica.

Capítulo 2

Fundamentos teóricos

2.1 De caracteres a estructura

El frente de un compilador puede representarse como una composición de funciones:

$$\text{caracteres} \xrightarrow{\text{lexer}} \text{tokens} \xrightarrow{\text{parser}} \text{árbol sintáctico}.$$

Esta separación reduce complejidad. El parser no necesita distinguir cada carácter de una palabra reservada; recibe un token como `IF` o `Identifier`. El lexer, a su vez, no decide si un `IF` aparece en una posición gramatical válida.

2.2 Análisis léxico

Un **lexema** es el fragmento concreto de entrada, por ejemplo `contador`; un **token** es su categoría, en este caso `Identifier`. Un patrón describe el conjunto de lexemas de una categoría. En Compiscript, un identificador comienza con letra o guion bajo y continúa con letras, dígitos o guiones bajos.

El lexer generado aplica prioridades y la política de coincidencia más larga. Por ello, las palabras reservadas se declaran antes de `Identifier`, y los operadores de dos caracteres se declaran antes que sus versiones de uno. Así, `<=` se reconoce como una unidad y no como `<` seguida de `=`.

Los espacios, saltos de línea y comentarios se envían a reglas con acción `skip`; no llegan al parser. Sin embargo, el lexer actualiza línea y columna para que los tokens y errores siguientes mantengan una ubicación correcta.

2.3 Gramáticas libres de contexto

Una gramática libre de contexto se describe como una cuádrupla $G = (V, \Sigma, P, S)$, donde V es el conjunto de no terminales, Σ el de terminales, P las producciones y S el símbolo inicial. En el proyecto, `program` es el símbolo inicial y exige cero o más sentencias seguidas de EOF:

`program : statement* EOF ;`

Los terminales son los tokens producidos por el lexer; los no terminales son reglas como `statement`, `functionDeclaration` o `expression`. El parser construye un árbol cuyos nodos internos corresponden a reglas y cuyas hojas corresponden a tokens.

2.4 Precedencia y asociatividad

Una gramática de expresiones debe evitar que `1 + 2 * 3` se interprete como `(1 + 2) * 3`. Compiscript separa cada nivel de precedencia en una regla. Cuanto más cerca está una regla de `primaryExpression`, mayor es su prioridad.

Cuadro 2.1: Precedencia de expresiones, de menor a mayor.

Nivel	Regla	Operadores o forma
1	Asignación	=
2	Condicional	? :
3	OR lógico	
4	AND lógico	&&
5	Igualdad	==, !=
6	Relacional	<, <=, >, >=
7	Aditiva	+, -
8	Multiplicativa	*, /, %
9	Unaria	!, -, +
10	Primaria	literales, identificadores, llamadas, índices y propiedades

2.5 ANTLR 4 y Adaptive LL(*)

ANTLR recibe una gramática declarativa y genera reconocedores para un lenguaje objetivo. En este laboratorio, `antlr4ts-cli` genera TypeScript. El parser de ANTLR 4 utiliza predicción **Adaptive LL(*)**: analiza la entrada de izquierda a derecha, construye una derivación izquierda y adapta su predicción con autómatas internos. Puede examinar la cantidad de contexto necesaria para distinguir alternativas.

No debe confundirse con un parser SLR. La implementación no posee tablas **ACTION/GOTO** ni registra operaciones *shift/reduce*. Su evidencia de ejecución es la invocación a las clases generadas y la construcción del árbol ANTLR.

2.6 Recuperación de errores

Detenerse en el primer error dificulta corregir un archivo grande. Una estrategia de recuperación intenta alcanzar un punto estable y continuar. La estrategia `DefaultErrorStrategy` de ANTLR puede:

- insertar conceptualmente un token faltante;
- eliminar un token sobrante;
- sincronizar el flujo con tokens esperados por reglas posteriores.

La recuperación no convierte una entrada inválida en válida: permite construir un árbol parcial y recolectar más diagnósticos. Es necesario controlar cascadas, porque un error inicial puede provocar mensajes secundarios que no representan problemas independientes.

2.6.1 Propagación y cascadas de errores

Cuando el lexer descarta un carácter no reconocido, el parser recibe un flujo de tokens distinto del texto original. Esa ausencia puede provocar un diagnóstico sintáctico secundario aunque el usuario haya cometido un solo error léxico. De forma similar, durante la recuperación sintáctica, la inserción o eliminación conceptual de un token puede modificar la interpretación de construcciones posteriores. A este fenómeno se le denomina *propagación o cascada de errores*.

La implementación no crea errores artificiales: conserva los diagnósticos producidos por ANTLR y aplica medidas para presentar únicamente información útil. Agrupa caracteres inválidos contiguos, elimina duplicados exactos, evita volver a reportar símbolos ya cubiertos por una cadena inválida y filtra errores del parser ubicados inmediatamente después de un fragmento reportado por el lexer. Las cascadas sintácticas lejanas no pueden clasificarse causalmente con certeza, por lo que se conservan hasta un máximo de cien diagnósticos por fase.

Capítulo 3

Especificación de Compiscript

3.1 Unidades léxicas

Cuadro 3.1: Familias principales de tokens.

Familia	Elementos representativos
Declaración	<code>let</code> , <code>var</code> , <code>const</code> , <code>function</code> , <code>class</code>
Control	<code>if</code> , <code>else</code> , <code>while</code> , <code>for</code> , <code>foreach</code> , <code>switch</code> , <code>try</code> , <code>catch</code>
Tipos	<code>integer</code> , <code>string</code> , <code>boolean</code> e identificadores de clase
Literales	enteros, cadenas, <code>true</code> , <code>false</code> , <code>null</code> y arreglos
Operadores	aritméticos, relacionales, igualdad, lógicos, asignación y ternario
Delimitadores	paréntesis, llaves, corchetes, coma, punto, dos puntos y punto y coma
Ignorados	espacios, tabulaciones, saltos de línea y comentarios de línea o bloque

Las cadenas admiten los escapes `\"`, `\\`, `\n`, `\r` y `\t`. Una comilla inicial sin cierre en la misma línea o una secuencia no permitida produce un diagnóstico léxico especializado.

3.2 Declaraciones y tipos

`let` y `var` permiten anotación de tipo e inicializador opcionales. `const` requiere inicializador. Los tipos básicos son `integer`, `string` y `boolean`; un identificador puede representar una clase. Cada sufijo `[]` agrega una dimensión de arreglo.

```
let contador: integer = 0;
var mensaje: string;
const ACTIVO: boolean = true;
let matriz: integer[][] = [[1, 2], [3, 4]];
```

3.3 Funciones y clases

Una función puede declarar parámetros tipados y un tipo de retorno. La gramática acepta recursión y funciones anidadas porque una declaración de función es una sentencia válida dentro de un bloque. Las clases admiten miembros, herencia mediante dos puntos, acceso con `this` e instanciación con `new`.

```
function factorial(n: integer): integer {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

```
class Perro : Animal {  
    function hablar(): string {  
        return this.nombre + " ladra.";  
    }  
}
```

3.4 Control de flujo

El lenguaje cubre bloques y ciclos `while`, `do-while`, `for` y `foreach`. La selección emplea `if/else` o `switch, case` y `default`; el manejo sintáctico usa `try/catch`. También existen `break`, `continue` y `return`. La palabra “sintáctico” es importante: el parser acepta la forma, pero no verifica, por ejemplo, que `break` aparezca semánticamente dentro de un ciclo.

3.5 Accesos y expresiones

La regla del lado izquierdo parte de un átomo primario y admite una sucesión de sufijos: llamada `(...)`, índice `[...]`, o propiedad `.identificador`. Esto permite reconocer construcciones como `obj.metodo(lista[0]).propiedad`. El agrupamiento con paréntesis y los niveles de la Tabla 2.1 preservan la estructura esperada.

Capítulo 4

Arquitectura y diseño

4.1 Arquitectura general

La lógica del analizador se concentra en `src/lib`; no está duplicada entre las interfaces. React y la CLI llaman a `analyzeInput`. Electron aloja la aplicación web ya compilada, por lo que también reutiliza el mismo núcleo.

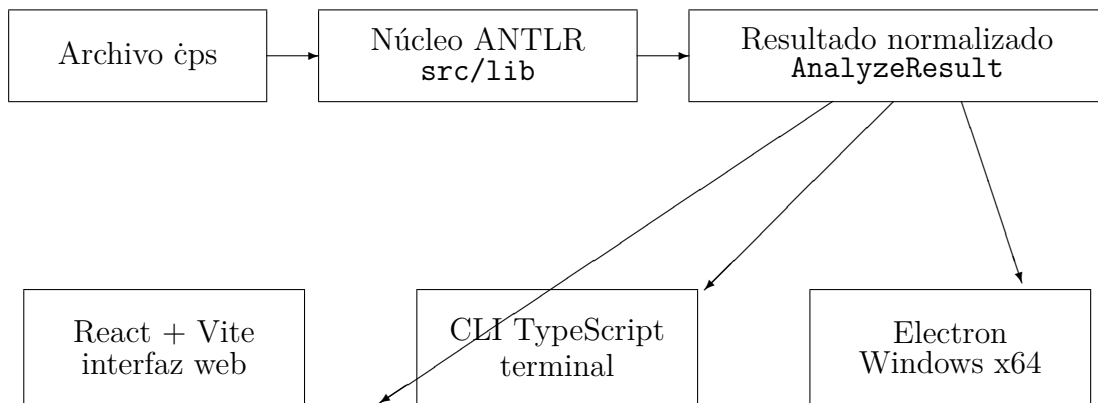


Figura 4.1: Arquitectura de reutilización del núcleo de análisis.

4.2 Generación e integración

La gramática es la fuente de verdad del lenguaje. El comando `npm run generate` ejecuta `antlr4ts` con opciones de visitor y listener, y escribe los artefactos en `src/generated`. Estos archivos se incluyen en el repositorio para que Java sea necesario únicamente al regenerarlos.

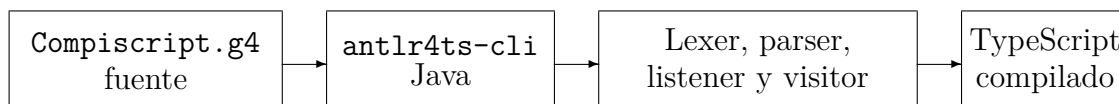


Figura 4.2: Flujo de generación de analizadores.

Los archivos generados no deben modificarse manualmente: una regeneración los sobrescribe. Para cambiar el lenguaje se edita primero la gramática, luego se genera de nuevo y finalmente se ejecutan las verificaciones.

4.3 Flujo en tiempo de ejecución

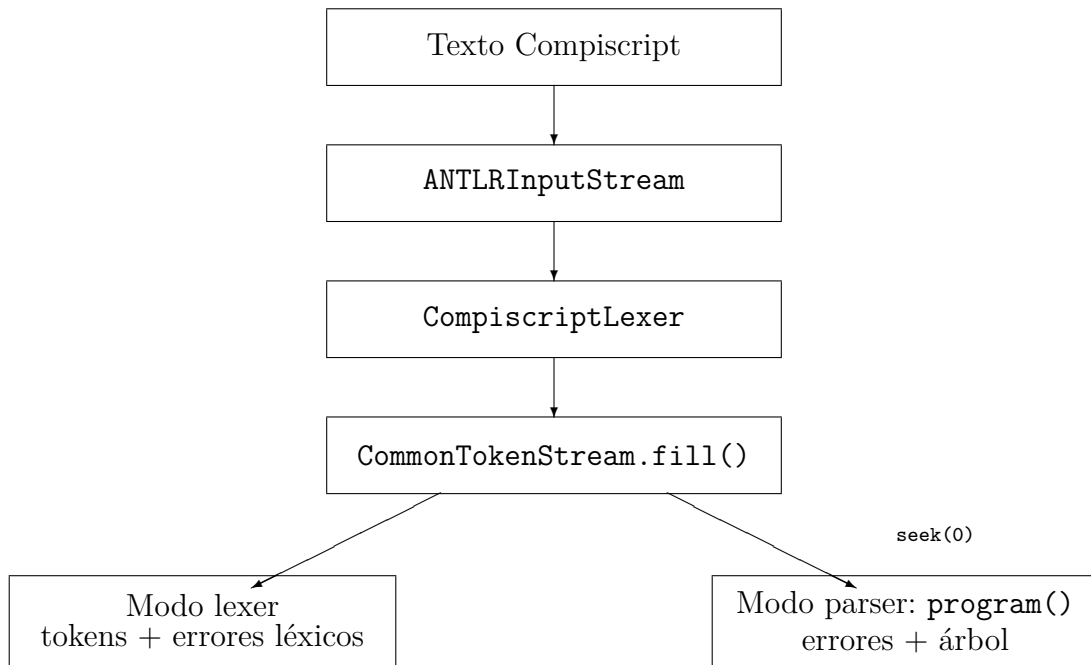


Figura 4.3: Pipeline de análisis y bifurcación por modo.

Los pasos concretos son:

1. Crear un `ANTLRInputStream` a partir del texto.
2. Instanciar `CompiscriptLexer` y sustituir sus listeners por uno recolector.
3. Crear `CommonTokenStream` y llamar `fill()` para forzar el recorrido completo.
4. Convertir cada token, excepto EOF, a `TokenInfo`.
5. Si el modo es léxico, terminar sin crear un parser.
6. En modo completo, rebobinar con `seek(0)` y crear el parser.
7. Configurar `DefaultErrorStrategy` e invocar `program`.
8. Convertir el árbol ANTLR a texto Lisp, texto indentado y nodos visuales.
9. Filtrar diagnósticos derivados y calcular la aceptación y el resumen.

4.4 Modelo de resultado

`AnalyzeResult` actúa como contrato entre el núcleo y sus consumidores. Contiene el lenguaje, modo, aceptación, tokens, errores por fase, tres formas del árbol, explicación y conteos. Cada token conserva índice, ID numérico, nombre simbólico, texto, línea, columna base 1 y canal. Cada error conserva origen, posición, mensaje, símbolo opcional y severidad.

Esta estructura desacopla el motor de la presentación. La UI puede renderizar tablas y árboles, la CLI puede imprimir texto y las funciones de descarga pueden serializar JSON o CSV sin volver a ejecutar el parser.

Capítulo 5

Implementación práctica

5.1 Organización del repositorio

```
.
|-- docs/                Documentación y matriz de cumplimiento
|-- electron/main.cjs    Proceso principal de Electron
|-- examples/
|   |-- compiscript/     Casos integrados de la interfaz
|   |-- rubric/low/      Cuatro casos de complejidad baja
|   |-- rubric/medium/   Cuatro casos de complejidad media
|   '-- error_distribution/
|       |-- lexer_errors.cps
|       |-- parser_errors.cps
|       '-- lexer_parser_alternating_errors.cps
|-- src/
|   |-- __tests__/       Pruebas Vitest
|   |-- cli/run.ts       Entrada de consola
|   |-- generated/       Código producido por ANTLR
|   |-- grammars/        Compiscript.g4
|   |-- lib/             Análisis, errores, árboles y descargas
|   |-- ui/              App y componentes React
|   |-- main.tsx         Punto de entrada web
|   '-- styles.css       Diseño adaptable
|-- package.json         Scripts y metadatos de empaquetado
|-- tsconfig.json        TypeScript estricto
'-- vite.config.ts       Vite, React y polyfills
```

5.2 Gramática combinada

Compiscript.g4 contiene reglas sintácticas y léxicas. La regla superior admite una lista de sentencias y exige llegar al final del archivo. Una versión reducida de la estructura es:

```
grammar Compiscript;

program
    : statement* EOF
    ;

statement
    : variableDeclaration
    | constantDeclaration
    | functionDeclaration
    | classDeclaration
    | printStatement
    | block
    | ifStatement
    | whileStatement
    | doWhileStatement
    | forStatement
    | foreachStatement
    | tryCatchStatement
    | switchStatement
    | breakStatement
    | continueStatement
    | returnStatement
    | expressionStatement
    ;
```

La expresión se divide por niveles en vez de usar un parser manual. La asignación permite un lado izquierdo seguido de = y otra asignación, o delega a la expresión condicional. La recursión a la derecha modela la asociatividad esperada de la asignación.

5.3 Integración en `analyze.ts`

La función pública acepta el texto y un modo opcional:

```
export function analyzeInput(  
  input: string,  
  mode: AnalyzerMode = "parser"  
): AnalyzeResult
```

El uso de `tokenStream.fill()` es una decisión central: fuerza al lexer a recorrer toda la entrada antes del parseo y hace posible capturar múltiples errores léxicos. Después se recolecta la información de los tokens, se descarta EOF y se obtiene cada nombre desde el vocabulario generado. En modo completo se ejecuta:

```
tokenStream.seek(0);  
const parser = new CompiscriptParser(tokenStream);  
parser.errorHandler = new DefaultErrorStrategy();  
parser.buildParseTree = true;  
const tree = parser.program();
```

La decisión de aceptación es explícita:

$$accepted \iff |E_{lex}| = 0 \wedge |E_{syn}| = 0.$$

Por tanto, producir tokens o un árbol parcial no implica que el archivo haya sido aceptado.

5.4 Recolección y normalización de errores

`CollectingErrorListener` implementa `ANTLRErrorListener` tanto para el lexer como para el parser. Convierte la columna interna base 0 a base 1 y genera un objeto uniforme. Los mensajes comunes se transforman como muestra la tabla.

Cuadro 5.1: Normalización de mensajes de ANTLR.

Patrón interno	Presentación al usuario
<code>token recognition error</code>	Carácter o lexema no reconocido
<code>missing X at Y</code>	Falta X antes de Y
<code>extraneous input</code>	Se encontró un token, pero se esperaba otro
<code>mismatched input</code>	El token no coincide con el conjunto esperado
<code>no viable alternative</code>	No se reconoció una construcción válida cercana
<code>failed predicate</code>	La construcción no cumple una regla de la gramática

El tratamiento adicional incluye:

- **Cadenas:** si una comilla no tiene cierre no escapado en la línea, se reporta una cadena sin cerrar; si sí existe cierre, el fragmento completo se reporta como cadena inválida por sus escapes.
- **Agrupación:** errores de reconocimiento contiguos en la misma línea se unen; `@@@` produce un diagnóstico y no tres.
- **Deduplicación:** no se agrega dos veces la misma combinación de línea, columna, símbolo y mensaje.
- **Cobertura:** no se reporta de nuevo un carácter que ya está dentro del fragmento de una cadena inválida.
- **Filtrado cruzado:** se omiten errores sintácticos inmediatamente cercanos a un fragmento ya descartado por el lexer.
- **Límite:** el centésimo elemento se convierte en un aviso de límite; el análisis continúa aunque se omitan mensajes posteriores.

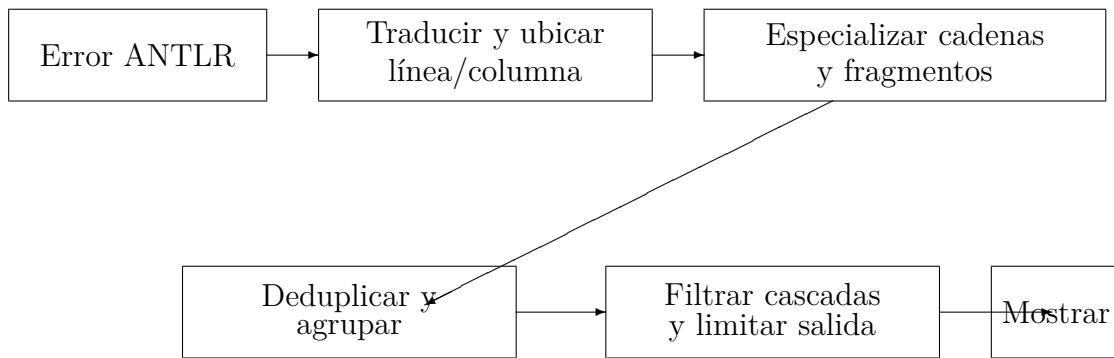


Figura 5.1: Proceso de normalización de diagnósticos.

5.5 Representación del árbol

El árbol se conserva de tres maneras:

1. `parseTreeText`: salida Lisp de `toStringTree`;
2. `formattedParseTree`: recorrido indentado para descarga y copia;
3. `parseTreeNodes`: nodos recursivos para la vista interactiva.

La conversión visual recorre directamente `ParseTree`. Si el nodo es un `ParserRuleContext`, usa el nombre de regla; si es terminal, usa su texto. Esta ruta evita ambigüedades que aparecerían al intentar volver a interpretar la cadena Lisp cuando un token literal es un paréntesis.

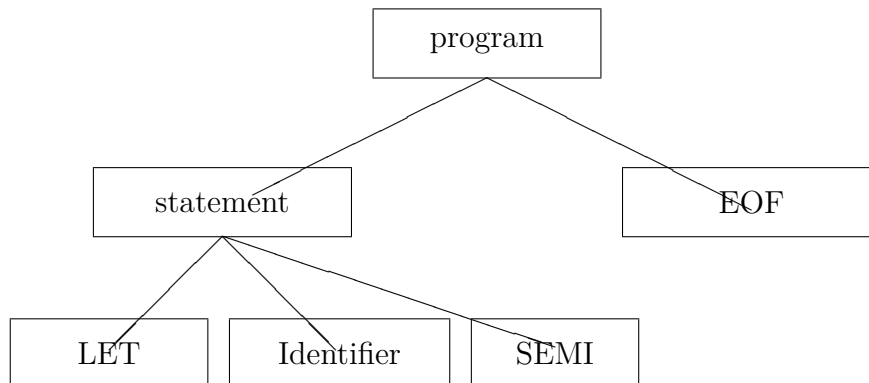


Figura 5.2: Ejemplo simplificado de árbol de parseo.

5.6 Interfaz web

`App.tsx` coordina estado, caso seleccionado, texto de entrada, pestaña, resultado y errores inesperados. La interfaz posee tres vistas:

- **Análisis completo:** lexer, parser, diagnósticos y árbol;
- **Solo lexer:** tokens y errores léxicos sin instanciar el parser;
- **Documentación:** cobertura, comandos y alcance.

`InputEditor` restringe el selector a `.cps`, valida nuevamente la extensión, lee el archivo como texto y permite editarlo. Los paneles presentan el estado, conteos, tabla de tokens, grupos de errores y árbol colapsable. La hoja de estilos utiliza rejillas y puntos de quiebre en 1100, 700 y 500 píxeles; los bloques de código, tablas y árbol poseen desplazamiento o ajuste de palabra para evitar desbordamientos.

Las descargas disponibles son gramática `.g4`, entrada `.cps`, tokens CSV, tokens JSON, árbol TXT en modo parser y resultado JSON. Se construyen con `Blob` y una URL temporal del navegador.

5.7 Compatibilidad con navegador

`antlr4ts` depende indirectamente de `assert`, `util` y la variable global `process`. Vite configura alias para los dos módulos y `browserProcess.ts` instala el polyfill global antes de cargar el analizador. Esta inicialización evita una pantalla en blanco por referencias de Node ausentes en el navegador.

5.8 Interfaz de consola

`src/cli/run.ts` valida que exista un argumento, que el archivo tenga extensión `.cpsy` que esté disponible. Lee UTF-8, llama al mismo `analyzeInput`, imprime conteos y errores con color, e informa si el análisis continuó después de errores recuperables.

Cuadro 5.2: Códigos de salida de la CLI.

Código	Significado
0	Archivo aceptado sin errores léxicos ni sintácticos
1	Archivo rechazado o argumentos de usuario inválidos
2	Excepción inesperada durante el análisis

5.9 Aplicación de escritorio

Electron crea una ventana de 1440×900 píxeles, con mínimos de 1000×700 , y carga `dist/index.html`. La configuración de seguridad desactiva la integración de Node, activa el aislamiento de contexto y mantiene el *sandbox*. El menú nativo se oculta.

`electron-builder` produce Windows x64 en dos modalidades:

`Compiscript-Portable-<version>.exe` `Compiscript-Setup-<version>.exe`

La aplicación final no requiere Node.js, npm ni Java en la computadora del usuario. Java solo se requiere al regenerar la gramática durante el desarrollo.

Capítulo 6

Pruebas y validación

6.1 Estrategia de pruebas

La suite utiliza Vitest y contiene 24 casos: 13 instancias en el archivo principal del analizador, 3 comprobaciones parametrizadas de sincronización entre los ejemplos de disco y los textos mostrados en la interfaz, y 8 comprobaciones parametrizadas de la matriz oficial de la rúbrica. Las pruebas no se limitan a verificar un valor booleano; también inspeccionan tokens posteriores, mensajes, conteos, estructuras exigidas y árbol.

Cuadro 6.1: Cobertura automatizada del proyecto.

Área	Evidencia verificada
Programa integral	Aceptación, cero errores, tokens de clases y foreach, y regla de función en el árbol.
Cobertura válida	Tipos, matrices, expresiones, ciclos, switch, try/catch, clases, herencia, new, closures y recursión.
Errores léxicos	Dos caracteres inválidos, continuidad hasta tokens posteriores, mensajes normalizados y ausencia de parser en modo lexer.
Agrupación	@@@ genera un solo error léxico y no una cascada sintáctica inmediata.
Cadenas	Diagnóstico específico para cadena sin cerrar y para escape inválido.
Límite léxico	120 fragmentos producen 100 entradas visibles y todavía se reconoce el identificador final.
Recuperación sintáctica	Se reporta más de un error, no se exponen frases internas en inglés y se conserva el árbol.
Límite sintáctico	Se limita la lista y el árbol alcanza EOF.
Sincronización	Los tres archivos .cpscoinciden con los casos embebidos en la interfaz.
Matriz de la rúbrica	Ocho archivos comprueban complejidad baja/media y los mínimos de errores por fase.

6.2 Casos demostrativos

El caso válido combina constantes, recursión, clases, herencia, arreglos, `foreach` y `try/catch`. Debe producir cero errores y un árbol desde `program` hasta EOF.

El caso léxico contiene dos símbolos no definidos y una sentencia posterior:

```
let primero: integer = 10 @ 2;
let segundo: integer = 20 # 4;
print(primeros);
```

La prueba espera dos errores y confirma que el token `print` todavía existe, lo cual demuestra continuidad del escaneo.

El caso sintáctico omite puntos y coma y un paréntesis de cierre:

```
let nombre: string = "Compiscript"
const total: integer = 10

if (total > 5 {
    print(nombre)
}
```

El parser debe recuperar más de un error y conservar una representación del árbol.

6.3 Matriz de casos exigidos por la rúbrica

La carpeta `examples/rubric` transforma cada criterio de evaluación en una entrada independiente. Los cuatro casos bajos contienen variables de tipo `integer`, `string` y `boolean`; una constante; suma y multiplicación; un condicional; un ciclo `while`; y un `foreach`. Los cuatro casos medios conservan esos elementos y agregan un arreglo declarado y utilizado, dos clases, dos objetos instanciados, dos funciones declaradas y dos llamadas.

Cuadro 6.2: Resultados verificados de la matriz de la rúbrica.

Caso	Tokens	Léxicos	Sintácticos	Salida
low/valid.cps	112	0	0	0
low/lexer_errors.cps	112	3	0	1
low/parser_errors.cps	109	0	3	1
low/lexer_parser_errors.cps	110	2	2	1
medium/valid.cps	263	0	0	0
medium/lexer_errors.cps	263	3	0	1
medium/parser_errors.cps	260	0	3	1
medium/lexer_parser_errors.cps	261	2	2	1

Los códigos de salida cero corresponden a aceptación sin errores. El código uno en los otros seis archivos confirma el rechazo intencional. Los casos exclusivamente léxicos no originan errores sintácticos derivados; los exclusivamente sintácticos no contienen símbolos fuera del vocabulario; y los mixtos alcanzan exactamente dos diagnósticos de cada fase. Todos presentan fase, línea, columna, símbolo o token y mensaje en español.

La automatización en `rubric.examples.test.ts` lee los archivos reales, ejecuta `analyzeInput`, verifica los conteos y comprueba por patrones la presencia de cada construcción de complejidad. Así se evita que la documentación y la demostración se separen del comportamiento actual.

6.4 Pruebas de distribución y recuperación de errores

Además de los tres casos integrados en la interfaz, el directorio `examples/error_distribution` contiene entradas manuales destinadas a observar propagación, recuperación y control de cascadas. Estos archivos funcionan como evidencia reproducible complementaria a las 24 pruebas Vitest para la interfaz gráfica, la CLI y el video.

Cuadro 6.3: Resultados verificados de los casos de distribución de errores.

Archivo	Tokens	Léxicos	Sintácticos	Total
lexer_errors.cps	39	3	0	3
parser_errors.cps	64	0	7	7
lexer_parser_alternating_errors.cps	72	4	4	8

6.4.1 Caso exclusivamente léxico

`lexer_errors.cps` contiene los símbolos inválidos `@`, `#` y `$`. El fragmento `###` se agrupa en un solo diagnóstico. El resultado final contiene tres errores léxicos, ningún error sintáctico derivado y tokens reconocidos después de cada fragmento incorrecto.

6.4.2 Caso exclusivamente sintáctico

`parser_errors.cps` utiliza únicamente caracteres reconocibles, pero omite puntos y coma y un paréntesis de cierre en construcciones independientes. El lexer no reporta problemas y el parser produce siete diagnósticos en español, alcanza EOF y conserva un árbol parcial mediante `DefaultErrorStrategy`.

6.4.3 Caso léxico y sintáctico alternado

`lexer_parser_alternating_errors.cps` intercala cuatro símbolos inválidos —`@`, `#`, `$` y un `&` aislado— con cuatro omisiones sintácticas deliberadas. El resultado contiene exactamente cuatro errores de cada fase. Los errores sintácticos inmediatos que podrían derivarse de los símbolos descartados no se duplican, y el análisis continúa hasta la sentencia final `print(e);`.

Los tres casos pueden ejecutarse desde la raíz del proyecto:

```
npm run cli -- examples/error_distribution/lexer_errors.cps
npm run cli -- examples/error_distribution/parser_errors.cps
npm run cli -- examples/error_distribution/lexer_parser_alternating_errors.cps
```

Todos terminan con código de salida 1 porque son archivos intencionalmente inválidos; ese código representa un rechazo esperado y no una excepción del analizador.

6.5 Comandos de verificación

La secuencia reproducible del proyecto es:

```
npm install
npm run generate
npm run check
npm test
npm run build
```

`npm run check` ejecuta TypeScript sin emitir archivos. `npm test` ejecuta Vitest una vez y `npm run build` combina comprobación de tipos y construcción optimizada de Vite. Estas tres verificaciones cubren tipos, comportamiento y empaquetado web, respectivamente.

Capítulo 7

Instalación, ejecución y distribución

7.1 Requisitos de desarrollo

- Node.js 20.19 o posterior, o 22.12 o posterior;
- npm 9 o posterior;
- Java/JRE 11 o posterior únicamente para regenerar los analizadores;
- navegador moderno; Windows x64 para los ejecutables distribuidos.

7.2 Ejecución web

En la raíz del repositorio:

```
npm install  
npm run generate  
npm run dev -- --force
```

Luego se abre <http://localhost:3000>. En ejecuciones posteriores normalmente basta `npm run dev`. La opción `-force` resulta útil si Vite debe reconstruir su caché después de instalar o cambiar dependencias.

7.3 Uso de la aplicación

1. Seleccionar **Análisis completo** o **Solo lexer**.
2. Elegir un caso incluido, escribir código o cargar un archivo **.cps**.
3. Presionar **Analizar con ANTLR**.
4. Revisar estado, tokens, ubicación de errores y, en modo completo, árbol.
5. Descargar la entrada o los resultados si se necesita evidencia externa.

7.4 Uso de la CLI

```
npm run cli:valid  
npm run cli:error-lex  
npm run cli:error-syn  
npx tsx src/cli/run.ts ruta/al/programa.cps
```

Los scripts de casos erróneos terminan intencionalmente con código 1; esto indica que la herramienta rechazó correctamente la entrada, no que el analizador falló.

7.5 Construcción de ejecutables

```
npm run exe:portable  
npm run exe:installer
```

Ambos comandos construyen primero la aplicación web y luego invocan **electron-builder**. Los artefactos quedan en **release**. Para una entrega normal se publica el portable o el instalador. Los metadatos **latest.yml**, los bloques **.blockmap**, los archivos internos NSIS y la carpeta **win-unpacked** no forman parte de la entrega para usuarios.

Windows SmartScreen puede advertir sobre el ejecutable porque el proyecto académico no posee un certificado comercial de firma. La procedencia debe comprobarse desde la sección oficial de Releases antes de elegir “Más información” y “Ejecutar de todas formas”.

Capítulo 8

Cumplimiento del laboratorio

Cuadro 8.1: Trazabilidad entre requisitos e implementación.

Requisito	Evidencia en el proyecto
Analizar Compiscript	Gramática <code>src/grammars/Compiscript.g4</code> y clases generadas.
Usar un generador	ANTLR 4 mediante <code>antlr4ts-cli</code> .
Recibir archivos <code>.cps</code>	Selector y validación de extensión en <code>InputEditor.tsx</code> ; validación equivalente en CLI.
Integrar lexer y parser	<code>analyzeInput()</code> ejecuta ambos en el modo principal.
Mostrar resultados	Resumen, tabla de tokens, grupos de errores y árbol textual/visual.
Clasificar errores	Listas independientes <code>lexicalErrors</code> y <code>syntaxErrors</code> .
Indicar ubicación	Listener con línea y columna base 1.
Indicar símbolo	Campo <code>offendingSymbol</code> visible y serializable.
Mensajes inteligibles	Traducción y especialización en <code>antlrErrors.ts</code> .
Continuar tras un error	Tokenización completa y <code>DefaultErrorStrategy</code> .
Evitar repeticiones	Deduplicación, agrupación, filtrado de cascadas y límite por fase.
Confirmar éxito	Banner de aceptación cuando ambas listas están vacías.
Interfaz amigable	React, componentes especializados y diseño adaptable.
Limitar el alcance	Sin semántica, ejecución ni generación de código.
Evidencia práctica	Ocho casos oficiales de la rúbrica, tres casos de distribución, tres casos integrados y 24 pruebas Vitest.

La trazabilidad muestra que los requisitos funcionales del laboratorio poseen una implementación identificable y evidencia automatizada o visible. La UI no sustituye al generador: presenta resultados provenientes de las clases creadas por ANTLR.

Capítulo 9

Resultados, limitaciones y trabajo futuro

9.1 Resultados obtenidos

El proyecto entrega un frente de compilador reutilizable en tres entornos. La gramática reconoce las construcciones previstas de Compiscript, el lexer conserva tokens posteriores a símbolos inválidos y el parser produce un árbol incluso después de recuperaciones posibles. La normalización hace que la salida sea más apropiada para una persona que aprende el lenguaje, sin ocultar la fase ni la ubicación del problema.

La separación entre motor, tipos y presentación facilita las pruebas y evita que las reglas de aceptación dependan de React o Electron. Los ejemplos sincronizados reducen el riesgo de que la demostración visual difiera de los archivos entregados.

9.2 Limitaciones actuales

- La corrección se define solo a nivel léxico y sintáctico.
- La recuperación puede producir un árbol parcial con tokens insertados o descartados conceptualmente; ese árbol no debe confundirse con ejecución.
- El límite de 100 diagnósticos prioriza una salida manejable sobre mostrar cada consecuencia de una entrada extremadamente dañada.
- El filtrado de cascadas usa cercanía de posición; es una mejora de presentación, no un análisis causal formal de errores.
- Los ejecutables configurados se dirigen a Windows x64.
- La firma comercial de código no forma parte del proyecto académico.

9.3 Trabajo futuro

Una evolución natural agregaría una tabla de símbolos y un visitor para comprobar declaraciones, ámbitos, herencia, llamadas y tipos. Después podría incorporarse un intérprete o una representación intermedia. También sería posible enriquecer los diagnósticos con fragmentos de fuente y subrayado, pruebas de propiedades para la gramática, medición de cobertura y empaquetado multiplataforma firmado.

Capítulo 10

Conclusiones

1. Una gramática declarativa permitió generar un lexer y parser reales y mantener separadas la definición del lenguaje y la interfaz de usuario.
2. Forzar la tokenización completa antes del parseo es esencial para recolectar múltiples errores léxicos y demostrar que el análisis continúa.
3. La recuperación predeterminada de ANTLR, acompañada de deduplicación, agrupación y filtrado, produce diagnósticos más útiles que detenerse en la primera anomalía o mostrar cascadas sin control.
4. La jerarquía de reglas de expresión implementa precedencia sin construir un parser manual ni usar tablas SLR.
5. Un contrato único de resultado hizo posible reutilizar el analizador desde React, la CLI y Electron, además de facilitar exportaciones y pruebas.
6. Las 24 pruebas automatizadas y la matriz de ocho casos de la rúbrica aportan evidencia de cobertura, recuperación, continuidad y consistencia.
7. El proyecto cumple el propósito del Laboratorio 01 dentro de su alcance léxico y sintáctico; las etapas semánticas y de generación quedan claramente separadas como trabajo posterior.

Referencias

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi y Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, segunda edición, 2006.
- [2] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, segunda edición, 2013.
- [3] ANTLR Project. *ANTLR 4 Documentation*. <https://www.antlr.org/>
- [4] Microsoft. *TypeScript Documentation*. <https://www.typescriptlang.org/docs/>
- [5] React Team. *React Documentation*. <https://react.dev/>
- [6] OpenJS Foundation. *Electron Documentation*. <https://www.electronjs.org/docs/>
- [7] DragonsSlayers 2.0. *Analizador de Compiscript: código fuente, README, gramática, pruebas y documentación técnica*. Repositorio del Laboratorio 01, 2026.

Apéndice A

Preguntas interesantes a las cuales se responde

¿El analizador es real? Sí. La función principal instancia el lexer y el parser de Compiscrypt generados por ANTLR e invoca `program`.

¿Cómo continúa después de errores? El lexer completa el flujo mediante `fill()` y el parser usa `DefaultErrorStrategy`; los listeners acumulan diagnósticos en lugar de lanzar la interfaz al primer error.

¿Cómo se evita repetir mensajes? Se deduplican coincidencias exactas, se agrupan caracteres contiguos, se omiten consecuencias sintácticas inmediatas de errores léxicos y se limita la vista por fase.

¿Por qué no hay tabla SLR? ANTLR 4 genera un parser Adaptive LL(*), por lo que no utiliza el algoritmo SLR ni sus tablas `ACTION/GOTO`.

¿Qué prueba la aceptación? Que no hubo errores léxicos ni sintácticos. No prueba corrección de tipos ni comportamiento en ejecución.

¿Por qué existe el código generado? Es el resultado reproducible de aplicar ANTLR a la gramática. Se versiona para ejecutar el proyecto sin exigir Java al usuario final, pero se regenera después de cambiar `Compiscrypt.g4`.

Apéndice B

Inventario de comandos

Comando	Propósito
<code>npm run generate</code>	Regenerar lexer, parser, listener y visitor.
<code>npm run dev</code>	Iniciar Vite en el puerto 3000.
<code>npm run check</code>	Verificar tipos TypeScript sin emitir archivos.
<code>npm test</code>	Ejecutar las 24 pruebas una vez.
<code>npm run build</code>	Validar tipos y crear dist .
<code>npm run cli:valid</code>	Analizar el ejemplo correcto desde terminal.
<code>npm run cli:error-lex</code>	Demostrar errores léxicos; salida esperada 1.
<code>npm run cli:error-syn</code>	Demostrar recuperación sintáctica; salida esperada 1.
<code>npm run cli - archivo.cps</code>	Analizar cualquier caso manual de la carpeta de ejemplos.
<code>npm run desktop</code>	Construir web y abrir Electron.
<code>npm run exe:portable</code>	Crear ejecutable portable Windows x64.
<code>npm run exe:installer</code>	Crear instalador NSIS Windows x64.

Apéndice C

Ficha técnica del producto

Elemento	Valor
Lenguaje analizado	Compiscript
Extensión de entrada	.cps
Gramática	src/grammars/Compiscript.g4
Generador	ANTLR 4 mediante antlr4ts-cli
Lenguaje objetivo	TypeScript
Frontend	React 19 y Vite 6
Pruebas	Vitest 3.2, 24 casos
Escritorio	Electron 43 y electron-builder 26
Plataforma empaquetada	Windows x64
Motor visible	ANTLR 4 / antlr4ts, código generado
Regla inicial	program
Recuperación	DefaultErrorStrategy
Máximo visible	100 diagnósticos por fase
Salidas	UI, consola, CSV, JSON, TXT y árbol visual
Alcance	Análisis léxico y sintáctico

DragonsSlayers 2.0

Laboratorio 01 – Construcción de Compiladores CC-3032

Documentación elaborada a partir de la versión actual del repositorio.